

Assignment 3:

- a. Create a Node.JS Application which serves a static website.
- b. Create four API using Node.JS, ExpressJS and MongoDB for CRUD Operations on assignment 2.C.

Theory:

Part a:

Create a Node.JS Application which serves a static website.

Installation : Node.js (site – Node.js), Express.js(installed through cmd)

Theory :

Node.js overview

In basic terms, Node is an open source cross-platform library for server-side programming that permits clients to develop web applications rapidly. With Node, we can execute JavaScript applications or network applications. Its basic modules are engraved in JavaScript.

It is generally utilized for server applications in real-time. Node.js permits JavaScript to execute locally on a machine or a server.

Node.js gives numerous systems to utilize. One of such structures is Express.js. It is more valuable and mainstream than the different structures of Node.js.

Features of Node.js

- **Versatility:** Node is incredibly adaptable as the server reacts in a non-blocking way.
- **Zero Buffering:** Applications yield the measurements in enormous pieces. This gives the advantage of 'No buffering' to developers.
- **Network:** Node.js upholds an open-source community. This is the main explanation that numerous glorious modules have been added to Node.js applications over time.
- **Occasion driven Input and output:** APIs of Node.js are non-blocking, meaning that the server won't wait for the arrival of information from an API. Rather, it will move to another API.

Advantages of Node.js

- **Easy to learn:** Node is quite simple for developers to utilize and learn. Learning Node.js is less difficult than React.
- **Better Performance:** Node.js takes the code of JavaScript via Google's V8 JavaScript engine. The main advantage of this process is that it complies with the JavaScript code directly into the machine code
- **Freedom:** Node.js offers a lot of freedom when it comes to development. There are generally less constraints with Node.
- **Extended support for tools:** Another advantage of Node.js is that developers have more community support.
- **Extensible:** Node.js is known to be quite extensible. You can utilize JSON to give the degree to trade of information between the web server and the client.
- **Scalability:** Node.js makes it simple to scale applications in horizontal as well as vertical directions. The applications can be scaled even by the option of extra hubs to the current framework.

Limitations of Node.js

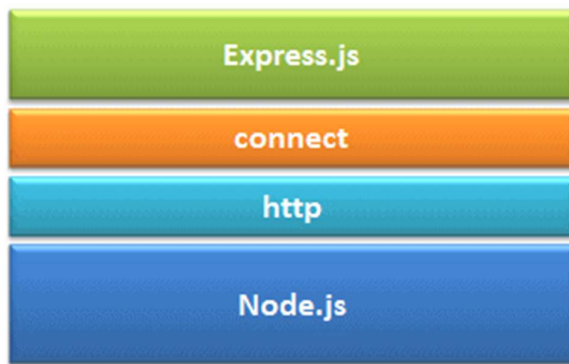
- **Programming interface isn't steady:** The Application Programming Interface (API) of Node can be challenging to work with. It changes regularly and doesn't remain stable.
- **No strong library support system:** JavaScript does not hold a strong library system. This limits the developers to implement even common programming tasks using Node.js.
- **Programming model is not synchronous:** Many developers find this programming model tougher in comparison to linear blocking I/O programming. In asynchronous programming, the codes become clumsier, and developers have to depend on the nes

Express.js

"Express is a fast, unopinionated minimalist web framework for Node.js" - official web site: Expressjs.com

Express.js is a web application framework for Node.js. It provides various features that make web application development fast and easy which otherwise takes more time using only Node.js.

Express.js is based on the Node.js middleware module called *connect* which in turn uses **http** module. So, any middleware which is based on connect will also work with Express.js.



Advantages of Express.js

1. Makes Node.js web application development fast and easy.
2. Easy to configure and customize.
3. Allows you to define routes of your application based on HTTP methods and URLs.
4. Includes various middleware modules which you can use to perform additional tasks on request and response.
5. Easy to integrate with different template engines like Jade, Vash, EJS etc.
6. Allows you to define an error handling middleware.
7. Easy to serve static files and resources of your application.
8. Allows you to create REST API server.
9. Easy to connect with databases such as MongoDB, Redis, MySQL

Steps :

1. Install Node.js
2. Setting up express.js
3. Structuring files
4. Creating your express server
5. Servicing your static files
6. Building your web page
7. Running your project

Open Node.js command terminal & run the following in your terminal-

Setting up express.js

1. Create a new directory for your project - mkdir **your-project-name**
2. Change into your new directory - cd **your-project-name**
3. Initialize a new Node project with defaults. This will set a package.json file to access your dependencies: npm init -y
4. Create your entry file, index.js. This is where you will store your Express server: if you are working on Linux, you can run : touch index.js. if you are working on windows, you can edit in VSCode
5. Install Express as a dependency : npm install express --save
6. Edit package.json. Within your package.json, update your start script to include node and your index.js file.

Let **express-static-file-tutorial** is your project name

Package.json

```
{
  "name": "express-static-file-tutorial",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node index.js" // change start value as node index.js
  },
  // This will allow you to use the npm start command in your
terminal to launch
  "keywords": [],
  "author": "Paul Halliday",
  "license": "MIT"
}
```

Structuring Your Files

To store your files on the client-side, create a public directory and include an index.html file

```
express-static-file-tutorial
|- index.js
|- public
```

|- index.html

Creating Your Express Server

Edit index.js file

Index.js

```
const express = require('express');
const app = express();
const PORT = 3000;

app.use(express.static('public')); // represents application is serving static webpage in public directory

app.get('/', (req, res) => {
  res.send('Hello World!');
});

app.listen(PORT, () => console.log(`Server listening on port: ${PORT}`));
```

First of all, import the Express.js module.

In the above example, we imported Express.js module using require() function. The express module returns a function. This function returns an object which can be used to configure Express application (app in the above example).

The app object includes methods for routing HTTP requests, configuring middleware, rendering HTML views and registering a template engine.

The app.listen() function creates the Node.js web server at the specified host and port. It is identical to Node's http.Server.listen() method. Instead of Get(), post(), put() and delete() methods can be used.

Building Your Web Page – client side

Navigate to your index.html file in the public directory. Populate the file with body and image elements:

[label index.html]

```
<html>
  <head>
    <title>Hello World!</title>
  </head>
  <body>
    <h1>Hello, World!</h1>
     //download & store image in public directory
  </body>
</html>
```

(Instead of building Hello world application, building applications like student's Registration form/main page of website is recommended)

Running Your Project

In your terminal, launch your Express project

npm start

It will display

Server listening on port : 3000

Open your web browser, and navigate to <http://localhost:3000>.

Conclusion

Assignment 4:

- a. Create a simple Mobile Website using jQuery Mobile.
- b. Deploy/Host Your web application on AWS VPC or AWS Elastic Beanstalk.

Mini Project

Develop a web application using full stack development technologies in any of the following domains:

1. Social Media
2. ecommerce
3. Restaurant
4. Medical
5. Finance
6. Education
7. Any other

Theory:

Part a:

jQuery Mobile

jQuery Mobile is a user interface framework, built on jQuery Core and used for developing responsive websites or applications that are accessible on mobile, tablet, and desktop devices. It uses features of both jQuery and jQueryUI to provide API features for mobile web applications. This tutorial will teach you the basics of jQuery Mobile framework. We will also discuss some detailed concepts related to jQuery Mobile.

Why Use jQuery Mobile?

- It creates web applications that it will work the same way on the mobile, tablet, and desktop devices.
- It is compatible with other frameworks such as PhoneGap, Whitelight, etc.
- It provides a set of touch-friendly form inputs and UI widgets.

Features of jQuery Mobile

- It is built on jQuery Core and "write less, do more" UI framework.
- It is an open source framework, and cross-platform as well as cross-browser compatible.
- It is written in JavaScript and uses features of both jQuery and jQuery UI for building mobile-friendly sites.

Download jQuery Mobile

When you open the link <https://jquerymobile.com/>, you will see there are two options to download jQuery mobile library.



Click the *Stable* button, which leads directly to a ZIP file containing the CSS and JQuery files, for the latest version of jQuery mobile library. Extract the ZIP file contents to a jQuery mobile directory.

This version contains all files including all dependencies, a large collection of demos, and even the library's unit test suite. This version is helpful to getting started.

Conclusion:

4.b Part b:

What is Cloud Computing?

Cloud computing is a term referred to storing and accessing data over the internet. It doesn't store any data on the hard disk of your personal computer. In cloud computing, you can access data from a remote server.

What is AWS?

The full form of AWS is Amazon Web Services. It is a platform that offers flexible, reliable, scalable, easy-to-use and, cost-effective cloud computing solutions.

AWS is a comprehensive, easy to use computing platform offered Amazon. The platform is developed with a combination of infrastructure as a service (IaaS), platform as a service (PaaS) and packaged software as a service (SaaS) offerings.

It is a secure cloud services platform, offering compute power, database storage, content delivery and other functionality to help businesses scale and grow.

In simple words AWS allows you to do the following things- Running web and application servers in the cloud to host dynamic websites.

History of AWS

- 2002- AWS services launched
- 2006- Launched its cloud products
- 2012- Holds first customer event
- 2015- Reveals revenues achieved of \$4.6 billion
- 2016- Surpassed \$10 billion revenue target
- 2016- Release snowball and snowmobile
- 2019- Offers nearly 100 cloud services
- 2021- AWS comprises over 200 products and services

Important AWS Services

Amazon Web Services offers a wide range of different business purpose global cloud-based products. The products include storage, databases, analytics, networking, mobile, development tools, enterprise applications, with a pay-as-you-go pricing model.

Applications of AWS services

Amazon Web services are widely used for various computing purposes like:

- Web site hosting
- Application hosting/SaaS hosting
- Media Sharing (Image/ Video)
- Mobile and Social Applications
- Content delivery and Media Distribution
- Storage, backup, and disaster recovery
- Development and test environments
- Academic Computing
- Search Engines
- Social Networking

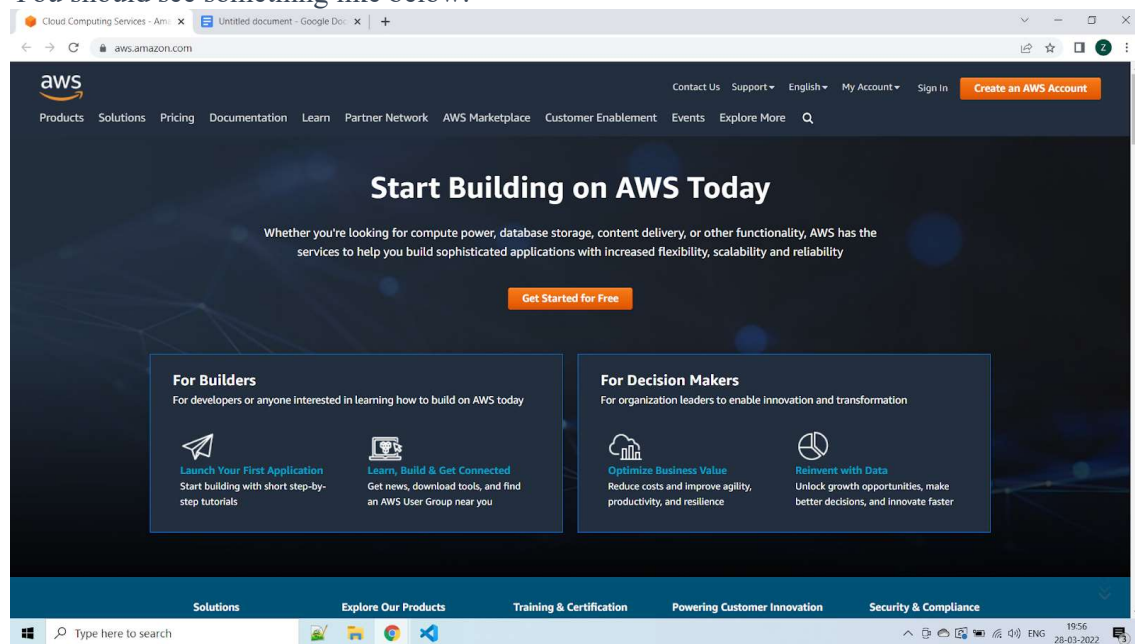
Creating an AWS Account is the first step you need to take in order to learn **Amazon Web Services**.

Steps to follow are as follows :

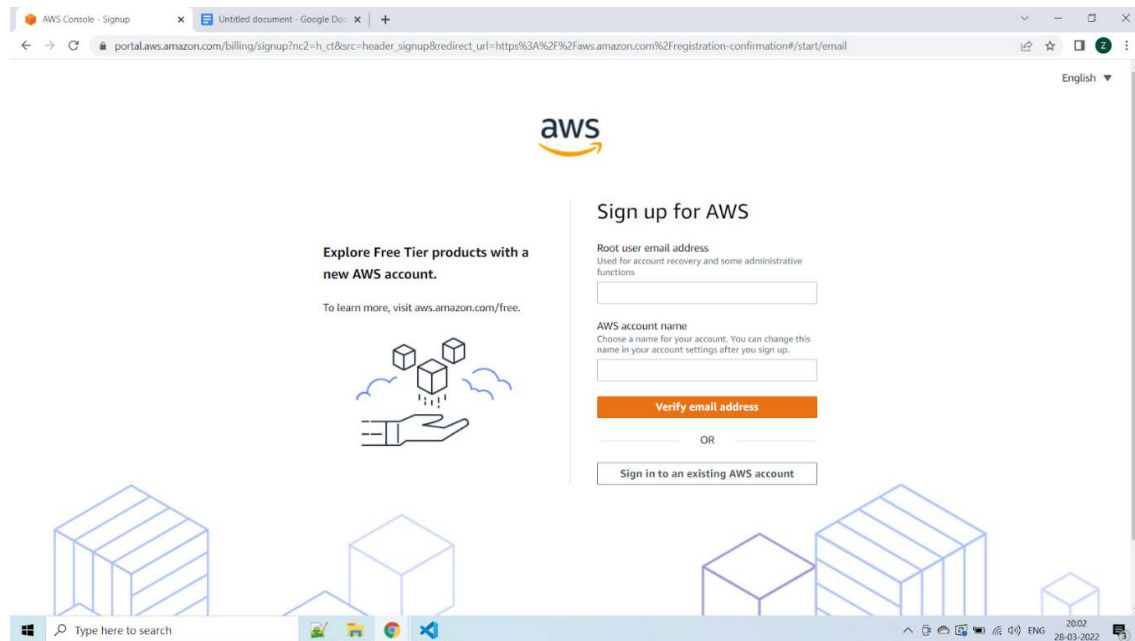
Step 1 – Visiting the Signup Page

Go to <https://aws.amazon.com>

You should see something like below:

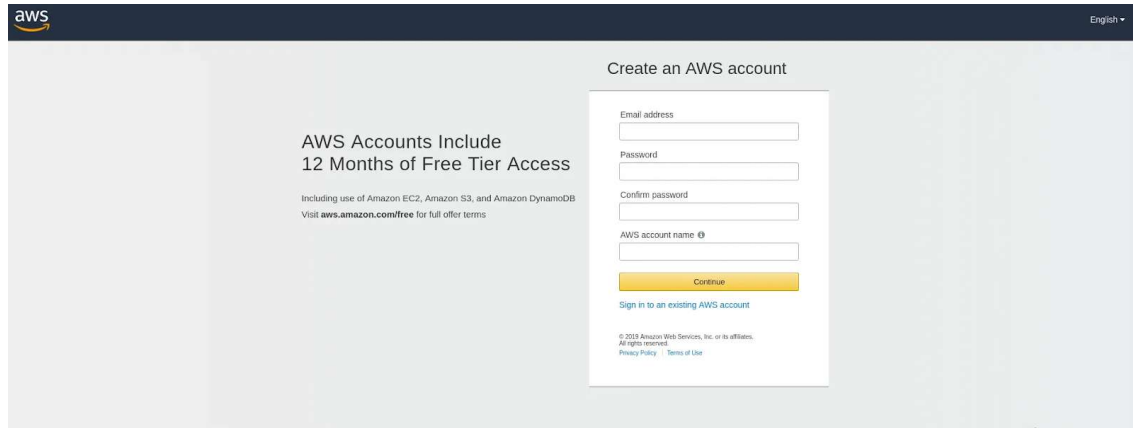


In order to continue, click the **Complete Sign Up** button in the middle of the screen or on the top right corner of the screen. You will see the below screen.



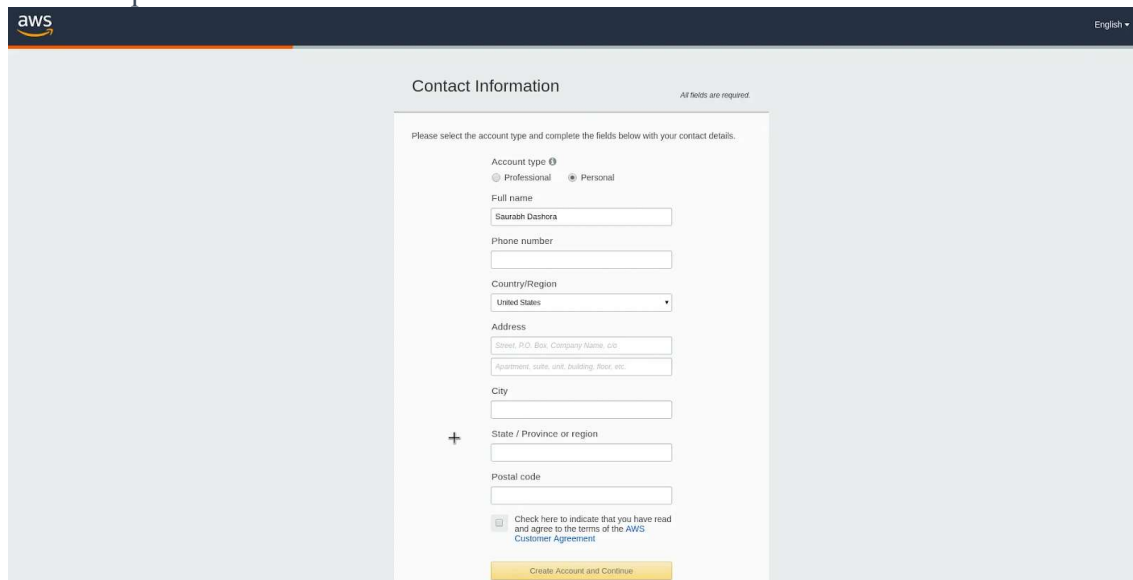
Step 2 – Entering User Details

After you have chosen to **Create a new AWS account**, you will see the below screen asking for few details.



You can fill up the details as per your requirements and click **Continue**.

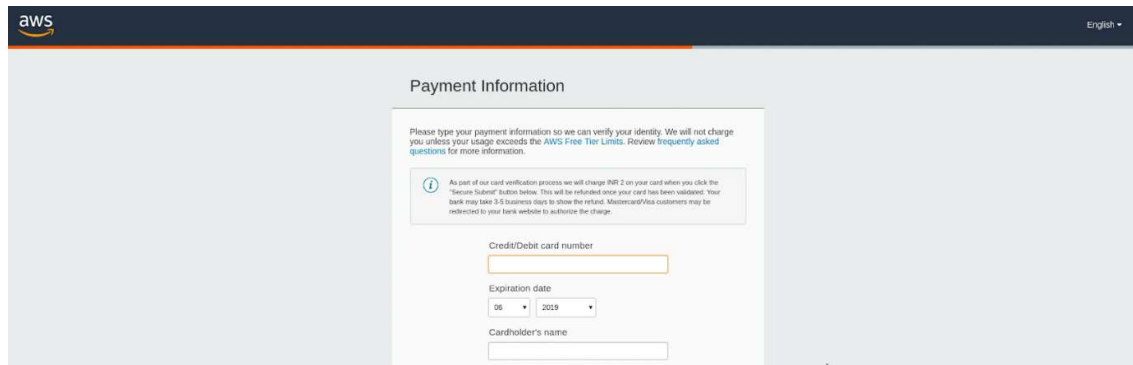
Next you will be asked to fill up your contact details such contact number, country, address and so on. You should fill them up properly because your contact number is important for further steps.



After filling up the details, click on the **Create Account and Continue** button at the bottom of the form.

Step 3 – Filling up the Credit Card details

For **Creating an AWS Account**, you need to enter your **Credit Card** details.

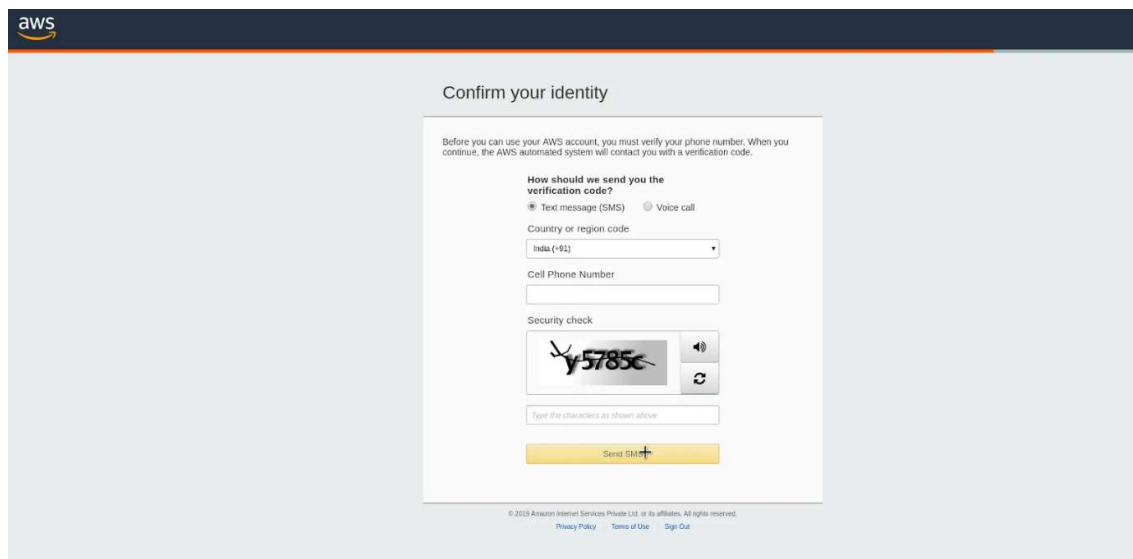


The screenshot shows the 'Payment Information' page on the AWS website. At the top left is the AWS logo, and at the top right is a language dropdown set to 'English'. The main heading is 'Payment Information'. Below it, a message states: 'Please type your payment information so we can verify your identity. We will not charge you unless your usage exceeds the AWS Free Tier Limits. Review frequently asked questions for more information.' An information icon (i) is next to this message. Below the message is a text box for 'Credit/Debit card number'. Underneath that is an 'Expiration date' section with two dropdown menus showing '06' and '2019'. At the bottom is a text box for 'Cardholder's name'.

After entering the details, click on **Secure Submit** button. It might take a while to process the request depending on your bank/credit card company servers.

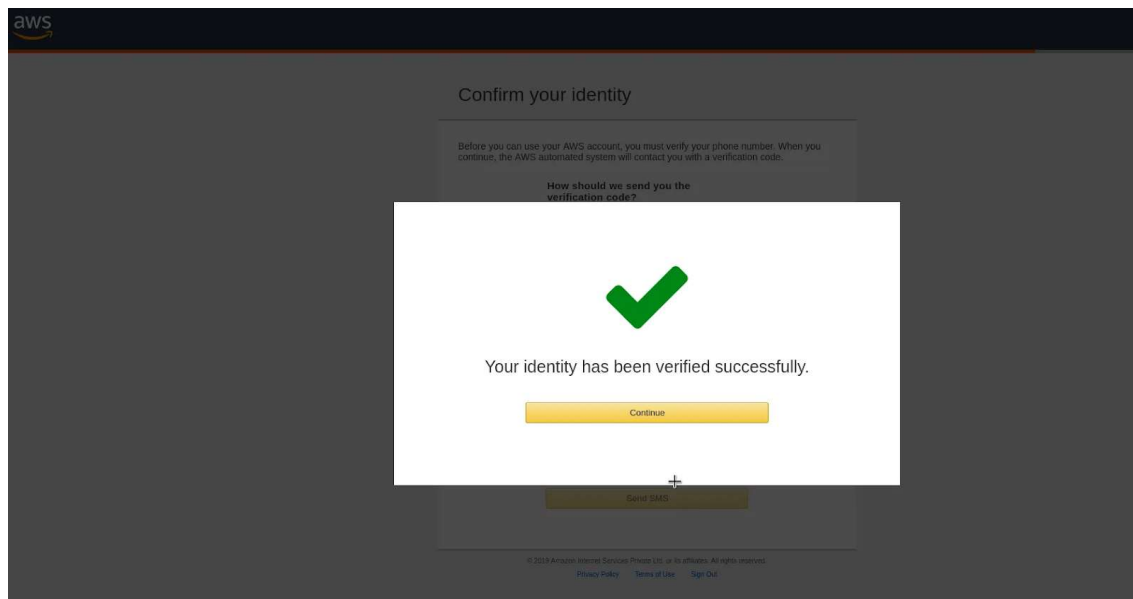
Step 4 – Identity Confirmation

Once the credit card details are confirmed, you will need to complete the **Identity Confirmation** step. You will see the below screen:

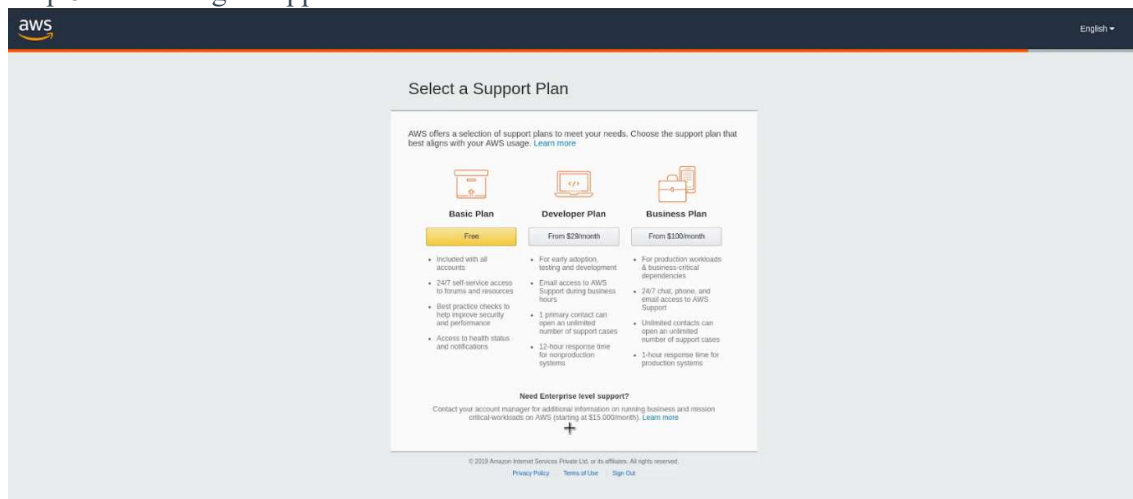


The screenshot shows the 'Confirm your identity' page on the AWS website. At the top left is the AWS logo, and at the top right is a language dropdown set to 'English'. The main heading is 'Confirm your identity'. Below it, a message states: 'Before you can use your AWS account, you must verify your phone number. When you continue, the AWS automated system will contact you with a verification code.' Below this message is a section titled 'How should we send you the verification code?'. It has two radio buttons: 'Text message (SMS)' (which is selected) and 'Voice call'. Below this is a 'Country or region code' dropdown menu showing 'India (+91)'. Underneath is a text box for 'Cell Phone Number'. Below that is a 'Security check' section. It features a CAPTCHA image showing the characters 'Y5785' and a 'Send SMS' button with a plus icon. At the bottom of the form is a small copyright notice: '© 2019 Amazon Internet Services Private Ltd. or its affiliates. All rights reserved.' with links for 'Privacy Policy', 'Terms of Use', and 'Sign Out'.

Once you have verified successfully, you should see a screen like below:

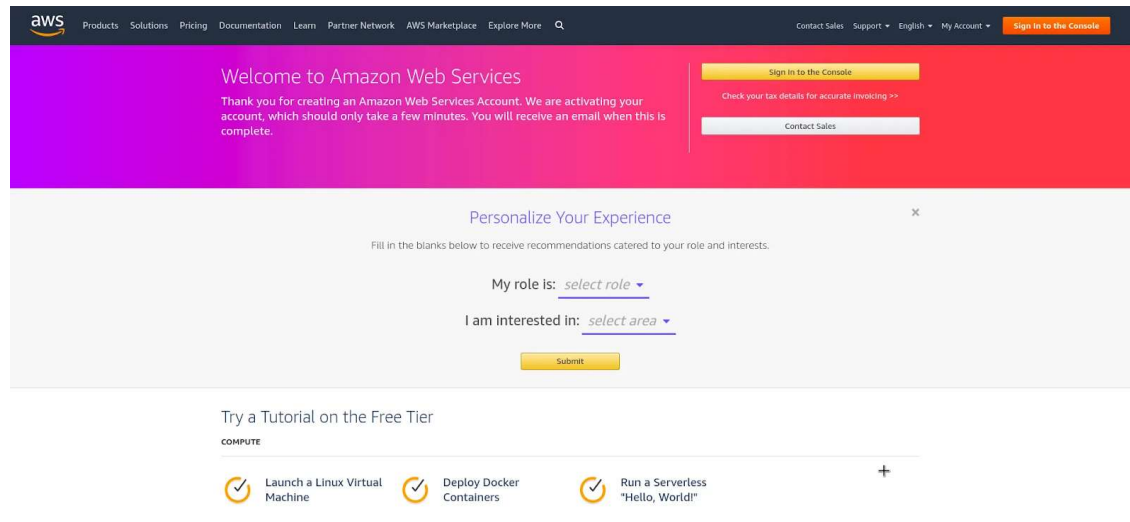


Step 5 – Selecting a Support Plan

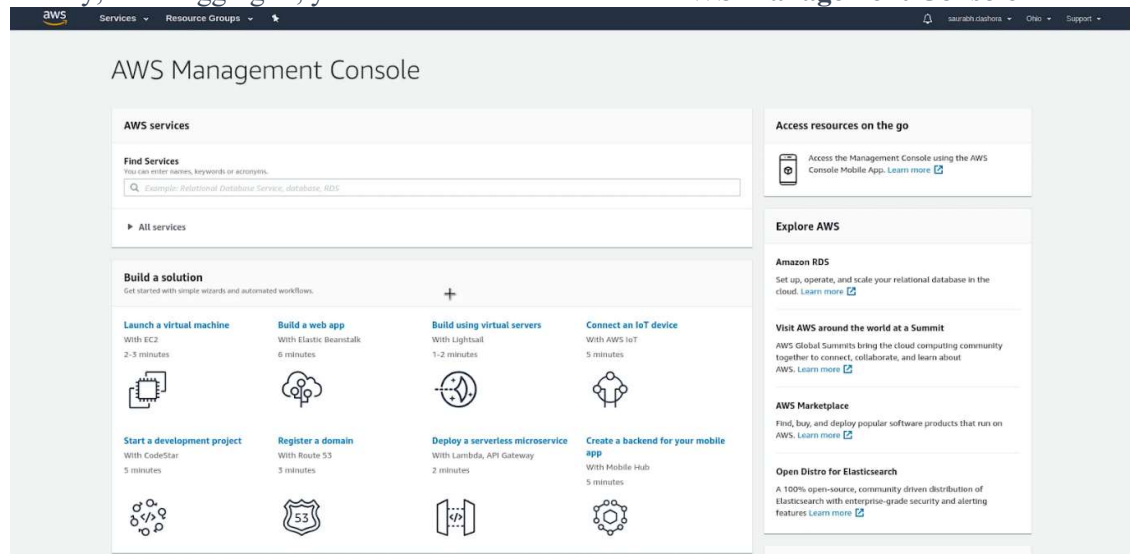


Go for **Basic Plan**. It is Free of cost and great for learning purposes. The other plans are **Developer Plan** and a **Business Plan**. But both of them are paid options.

Once you select your plan, you will see the below **Welcome** screen. From here on, you can Sign in to your **AWS Console**.



Finally, after logging in, you should be able to see the **AWS Management Console** as below:



If you have reached this far, you have successfully finished **Creating an AWS Account**.

Deployment Steps:

Step 1: Launch a Windows Server Amazon EC2 instance.

Step 2: Configure your source content to deploy to the Windows Server Amazon EC2 instance.

Step 3: Upload your "hello, world!" ...

Step 4: Deploy your Hello World application.

Step 5: Update and redeploy your "hello, world!" ...

Step 6: Clean up your "hello, world!"

Conclusion :